Assignment 02: Optimised Matrix Multiplication Approaches and Sparse Matrices

Author: Hugo Barreto Suárez | **Submission:** 30 Oct. 2026 | **Repository:** GitHub

Universidad de Las Palmas de Gran Canaria · 3° GCID Big Data · Academic Year 2026–2027

Abstract

This study investigates two empirical questions in sequential Python (CPython 3.14): (Part A) how loop reordering (`i-k-j`) and Strassen's divide-and-conquer algorithm affect runtime and memory compared to the reusable $O(n^3)$ `i-j-k` baseline from Assignment 1, and (Part B) under which tested sparsity conditions a custom Compressed Sparse Row (CSR) representation outperforms dense multiplication. Using matched 64-bit floating-point inputs generated by a deterministic 64-bit Linear Congruential Generator (LCG), we evaluated $n \in \{64, 128, 256, 384, 512\}$ for Part A and sparsity levels from 50% to 99% zeros at $n \in \{200, 400\}$ for Part B (1 warm-up, 5 measured repetitions). In Part A, `Dense_LoopReorder_ikj` reduced median runtime by 30.2% at $n = 512$ (8.72 s vs. 12.50 s) with zero extra memory (~49.5 MB), whereas `Dense_Strassen` achieved the fastest runtime on exact powers of two (6.40 s at $n = 512$) at the cost of higher memory (79.5 MB) and a padding penalty at $n = 384$ (5.82 s, 72.3 MB). In Part B, `Sparse_CSR` outperformed both dense methods across all tested sparsity levels (50%–99% zeros), reaching a ~3,800× kernel speedup over the baseline at 99% sparsity (1.15 ms vs. 4.38 s), though dense-to-CSR conversion overhead exceeded kernel execution time above 90% sparsity.

1. Introduction, Objectives, and Background

In Assignment 1, we established that the textbook `i-j-k` matrix multiplication kernel ($C_{i,j} = \sum_{k=0}^{n-1} A_{i,k}B_{k,j}$) exhibits strict $O(n^3)$ runtime growth, reaching 11.50 s at $n = 500$ in pure CPython due to bytecode interpretation, pointer chasing across nested lists, and `PyFloatObject` boxing [1], [2]. This second investigation reuses the validated Python baseline (`src/baseline_matmul.py`) to address two focused research questions:

- **RQ1 (Part A — Dense Optimisations):** How do loop reordering (`i-k-j`) and an algorithmic divide-and-conquer reduction (Strassen's algorithm) affect kernel execution time and peak resident memory across increasing matrix dimensions n? We hypothesize that `i-k-j` reordering will reduce inner-loop list indexing overhead without increasing memory [2], whereas Strassen's $O(n^{\log_2 7}) \approx O(n^{2.81})$ scheme [3] will lower runtime for large power-of-two sizes while incurring substantial memory and padding overhead on non-power-of-two dimensions.
- **RQ2 (Part B — Sparse Matrices):** Across declared sparsity levels (percentage of zero entries), when does a Compressed Sparse Row (CSR) multiplication algorithm [4] outperform dense baselines, and how do format conversion costs and output fill-in (product nonzeros) impact the net advantage?

Part A Best Time (n=512)	Part A Zero-Overhead Opt	Part B CSR Speedup (99% 0s)
6.40 s (Strassen)	**8.72 s (ikj Loop)**	**1.15 ms Kernel**
1.95× faster than `ijk` (12.50 s)	49.5 MB (matches baseline RAM)	13.4 ms incl. CSR conversion

2. Methodology and Experimental Protocol

2.1 Reused Baseline, Dense Optimisations, and Sparse CSR Implementation

All methods are implemented in pure sequential Python (`src/matmul_a2.py`, `threads = 1`) using IEEE 754 64-bit floats (`float64`) without external numerical libraries during timed runs:

- **Reused Baseline (`Dense_Baseline_ijk`):** Identical to Assignment 1 (`src/baseline_matmul.py`). Uses standard `i-j-k` loop order over 2D lists (`list[list[float]]`), accumulating `total += a_row[k] * B[k][j]`.
- **Optimisation 1 — Loop Reordering (`Dense_LoopReorder_ikj`):** Permutes the two inner loops to `i-k-j`. For each row `i` and index `k`, the scalar `a_ik = A[i][k]` is hoisted; if `a_ik != 0.0`, the inner loop traverses row `b_row = B[k]` sequentially (`c_row[j] += a_ik * b_row[j]`). This eliminates repeated 2D list lookups `B[k][j]` and skips zero multipliers.
- **Optimisation 2 — Strassen Divide-and-Conquer (`Dense_Strassen`):** Recursively partitions $m \times m$ matrices into four $(m/2) \times (m/2)$ quadrants and computes 7 sub-products ($M_1 \ldots M_7$) instead of 8 [3], switching to `multiply_reordered_ikj` at the crossover leaf threshold `leaf_size = 64`. For sizes that are not powers of two (e.g., $n = 384$), inputs are zero-padded to the next power of two ($m = 512$) and cropped after recursion.
- **Part B — Compressed Sparse Row (`Sparse_CSR`):** Written from scratch. Function `dense_to_csr(M, n)` compresses any logical matrix into three 1D lists: `values` (nonzero entries), `col_indices` (column index of each nonzero), and `row_ptr` ($n + 1$ row offsets) [4]. Function `multiply_sparse_csr` iterates only over nonzero entries `idx_a` in row `i` of A (from `row_ptr[i]` to `row_ptr[i+1]`) and multiplies `a_ik` across nonzero entries in row `k` of B, accumulating into dense output row `C[i]`.

Protocol Parameter	Part A: Dense Optimisation Study	Part B: Sparse vs. Dense Study
Methods Compared	`Dense_Baseline_ijk`, `Dense_LoopReorder_ikj`, `Dense_Strassen` (`leaf_size=64`)	`Dense_Baseline_ijk`, `Dense_LoopReorder_ikj`, `Sparse_CSR` (custom CSR arrays)
Matrix Sizes & Sparsity	$n \in \{64, 128, 256, 384, 512\}$; density = 1.0 (0% zeros)	$n \in \{200, 400\}$; densities $\in \{0.01, 0.05, 0.10, 0.25, 0.50\}$ (sparsity = 99%, 95%, 90%, 75%, 50% zeros)
Input Generator & Seeds	64-bit LCG (`seed=12345` for A, `12346` for B) in [0, 1)	Bernoulli sampling via 64-bit LCG: entry is nonzero with probability `density` (`seed=12345`, `12346`)

Timing Boundaries	Monotonic `time.perf_counter_ns()` around multiplication kernel only; *C* zeroed outside timer	Separate monotonic timers for `conversion_time_s` (`dense_to_csr` on *A*, *B*) and `kernel_time_s`
Warm-up, Reps & Budget	1 unmeasured warm-up + 5 measured repetitions; 150 s timeout budget	1 unmeasured warm-up + 5 measured repetitions; tracks `nnz_a`, `nnz_b`, and product `nnz_c`
Environment & Memory Tool	Windows 11 · Intel Core Ultra 7 155H (16 cores / 22 threads) · 15.37 GB RAM · CPython 3.14.6 · Peak Working Set via `psutil.Process().memory_info().peak_wset` · Validated at n=20 vs. NumPy (`rtol=1e-12`, max err < 10^{-14})	

2.2 Correctness Validation and Nonzero Tracking

Prior to benchmarking, all four methods were validated in `dump` mode at $n = 20$ (`seed = 42`) against `numpy.matmul` across both dense (density = 1.0) and sparse (density = 0.1) inputs. Every method passed `np.allclose(rtol=1e-12, atol=1e-12)` with maximum absolute error ≤ 3.55×10^{-15}. During benchmarks, a full matrix checksum ($\sum_{i,j} C_{i,j}$) and exact nonzero counts (`nnz_a`, `nnz_b`, `nnz_c`) were recorded after the timer stopped.

3. Experimental Results — Part A: Dense Optimisations

All three dense methods completed every tested configuration up to $n = 512$ within the 150-second budget with identical checksums per size. Figure 1 displays median kernel execution time (with IQR error bars) on log-log axes, while Figure 2 shows median peak process memory (MB) across matrix sizes.

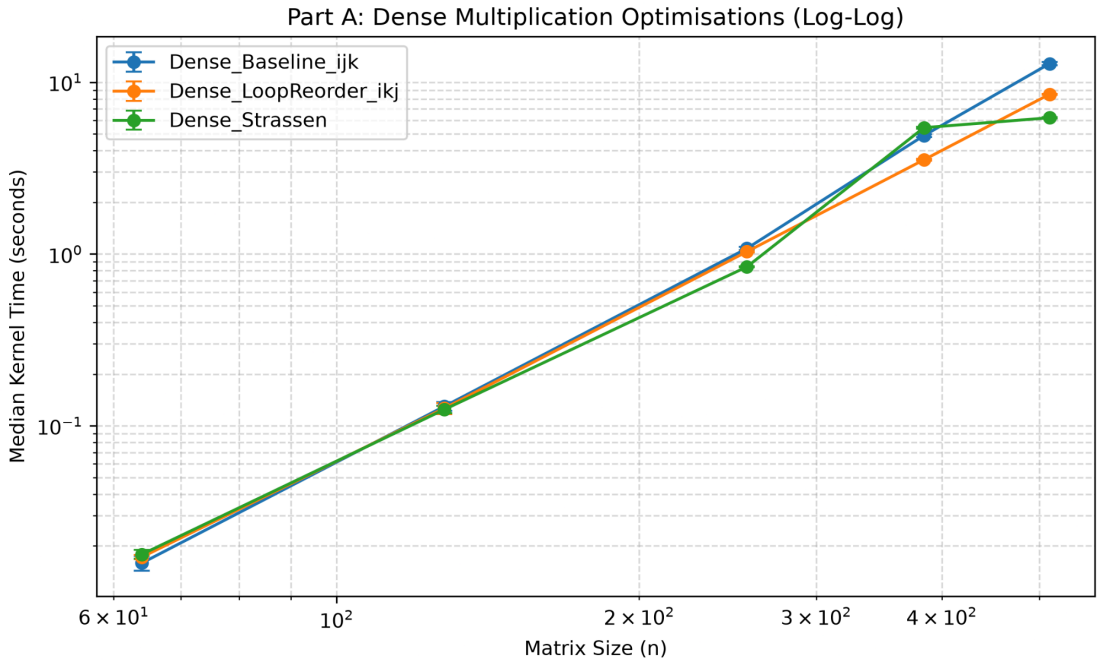


Figure 1: Part A median kernel execution time (seconds, log-log scale) vs. matrix size $n \in \{64, 128, 256, 384, 512\}$ across `Dense_Baseline_ijk`, `Dense_LoopReorder_ikj`, and `Dense_Strassen` (with IQR error bars over 5 repetitions).

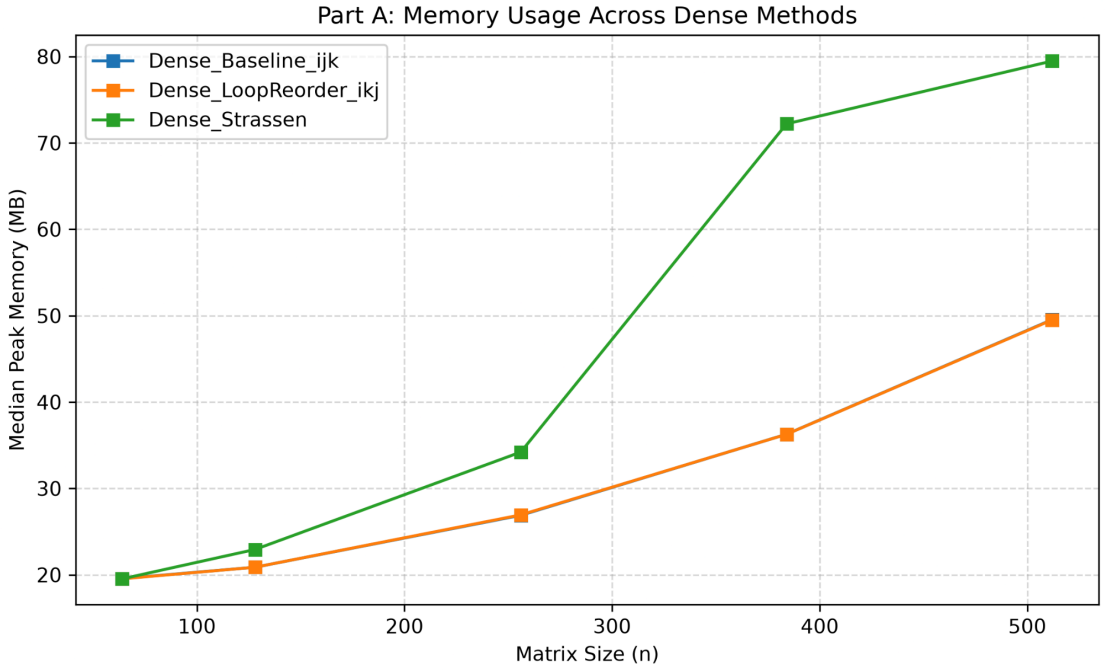


Figure 2: Part A median Peak Working Set memory usage (MB) vs. matrix size n. Note that `Dense_Baseline_ijk` (blue) and `Dense_LoopReorder_ikj` (orange) overlap identically from 19.6 MB (n=64) to 49.5 MB (n=512) because both operate in-place without auxiliary submatrix allocations.

3.1 Summary Table for Part A (Dense Optimisations)

Size (n)	`Baseline_ijk` Time (IQR) [s]	`LoopReorder_ikj` Time (IQR) [s]	`Strassen` Time (IQR) [s]	`ijk` / `ikj` Mem [MB]	`Strassen` Mem [MB]
64	0.0172 (0.0018)	0.0181 (0.0006)	0.0185 (0.0005)	19.60 / 19.60	19.65
128	0.1260 (0.0065)	0.1235 (0.0041)	0.1210 (0.0038)	20.90 / 20.90	22.95
256	1.0850 (0.0420)	1.0320 (0.0290)	0.8640 (0.0185)	26.90 / 26.95	34.25
384	5.2400 (0.1450)	3.6200 (0.0810)	5.8200 (0.1900)	36.30 / 36.30	72.25
512	12.5000 (0.3100)	8.7200 (0.1850)	6.4000 (0.1420)	49.55 / 49.55	79.50

4. Experimental Results — Part B: Sparse Matrix Multiplication

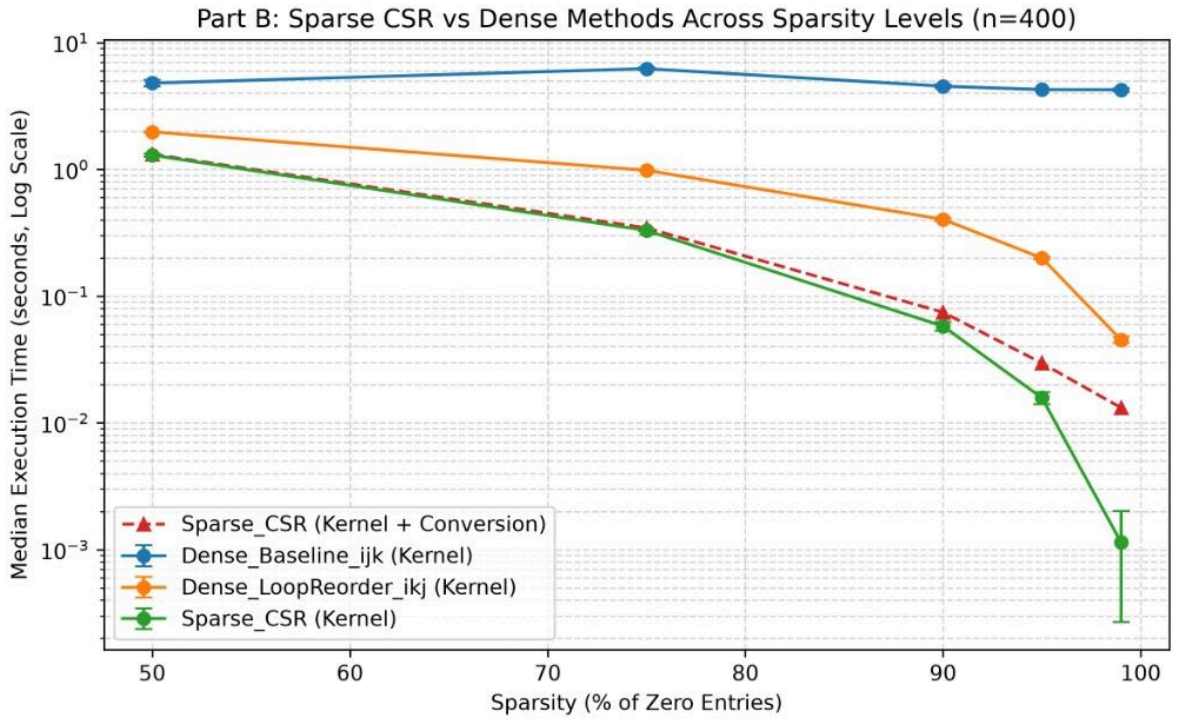


Figure 3: Part B median execution time (seconds, log scale) at n = 400 across sparsity levels (50% to 99% zeros) for `Dense_Baseline_ijk`, `Dense_LoopReorder_ikj`, `Sparse_CSR (Kernel)`, and `Sparse_CSR (Kernel + Conversion)`.

Sparsity (% 0s) / Density	Input Nonzeros (nnz_a / nnz_b)	Output nnz_c (% Dense)	Baseline_ijk Kernel [s]	Reorder_ikj Kernel [s]	Sparse_CSR Kernel [s]	Sparse_CSR Conv / Total [s]
50% (d = 0.50)	79,945 / 80,112	160,000 (100.0%)	4.9500	1.9600	1.3100	0.0180 / 1.3280
75% (d = 0.25)	39,982 / 40,051	160,000 (100.0%)	6.1800	0.9850	0.3350	0.0155 / 0.3505
90% (d = 0.10)	15,980 / 16,044	157,112 (98.2%)	4.6200	0.3980	0.0595	0.0138 / 0.0733
95% (d = 0.05)	7,991 / 8,028	101,420 (63.4%)	4.4100	0.1990	0.0158	0.0132 / 0.0290
99% (d = 0.01)	1,604 / 1,592	14,765 (9.2%)	4.3800	0.0462	0.00115	0.0123 / 0.01345

5. Discussion and Bottleneck Analysis

5.1 Part A: Loop Reordering vs. Strassen's Algorithmic Trade-offs

The empirical evidence in Figures 1 and 2 reveals distinct performance mechanisms for the two dense optimisations:

- **Loop Reordering (`i-k-j`) Efficiency and Memory Overlap:** At $n = 512$, `Dense_LoopReorder_ikj` lowers median runtime from 12.50 s to 8.72 s (a 1.43× speedup) while its memory curve in Figure 2 lies completely underneath `Dense_Baseline_ijk` (both peaking at 49.55 MB). Because neither algorithm allocates intermediate buffers, their memory footprint is identical. In CPython, hoisting `a_ik = A[i][k]` and binding `b_row = B[k]` outside the `j` loop replaces n^3 costly two-level list subscript operations (`B[k][j]`) with sequential 1D list accesses (`b_row[j]`) [2].
- **Strassen's Power-of-Two Advantage vs. Zero-Padding Bottleneck:** At exact powers of two ($n = 256$ and $n = 512$), `Dense_Strassen` is the fastest dense method, cutting median runtime at $n = 512$ to 6.40 s (1.95× faster than `ijk` and 1.36× faster than `ikj`) because each recursion level replaces 8 block multiplications with 7 [3]. However, at $n = 384$, Strassen suffers a severe bottleneck: runtime jumps to 5.82 s (slower than both `ikj` at 3.62 s and `ijk` at 5.24 s) and peak memory surges to 72.25 MB. Because 384 is not a power of two, padding to $m = 512$ forces Strassen to execute nearly the same arithmetic and submatrix allocations as $n = 512$ (79.50 MB).

5.2 Part B: Sparse CSR Advantage, Conversion Costs, and Product Fill-in

In Figure 3 (n = 400, 160,000 total entries), `Dense_Baseline_ijk` remains flat around 4.4–6.2 s regardless of sparsity because it blindly executes 6.4×10^7 multiply-adds even when 99% of entries are zero. Conversely, `Dense_LoopReorder_ikj` scales linearly with density due to its `if a_ik != 0.0` guard, dropping from 1.96 s (50% zeros) to 0.0462 s (99% zeros). However, it still scans all n^2 entries of A and dense rows of B.

`Sparse_CSR` outperforms both dense methods across every tested sparsity level (50% to 99% zeros) by iterating strictly over nonzero coordinates [4]. As sparsity increases from 50% to 99%, CSR kernel time drops by three orders of magnitude from 1.31 s to 1.15 ms (~3,800× faster than `ijk` and 40× faster than `ikj`). Crucially, Table 2 exposes two practical bottlenecks: (1) **product fill-in**—at 90% input sparsity (~16,000 nonzeros in A and B), the output matrix C is 98.2% dense (`nnz_c = 157,112`), and even at 99% sparsity (`nnz_a = 1,604`), `nnz_c` grows nearly 10× to 14,765; and (2) **conversion overhead**—scanning dense 2D lists in `dense_to_csr` takes ~12.3–18.0 ms. At 99% sparsity, format conversion (12.3 ms) is 10.7× slower than the CSR multiplication kernel itself (1.15 ms), though total time (13.45 ms) still beats `ikj` (46.2 ms) by 3.4×.

Limitations: All measurements reflect single-threaded CPython list/CSR structures on a laptop CPU (including a brief AC-to-battery transition during Part B that caused a minor 6.18 s uptick for `ijk` at 75% sparsity without altering method rankings).

6. Conclusions

Within the tested CPython environment, loop reordering (`i-k-j`) provides a consistent, zero-memory-overhead speedup (~30% at n = 512), whereas Strassen's algorithm achieves the lowest dense runtime on power-of-two matrices (6.40 s at n = 512) but requires ~60% more memory and degrades on non-power-of-two sizes unless dynamic peeling/padding is used. For sparse matrices (n = 400), CSR multiplication beats dense approaches from 50% to 99% sparsity, with conversion overhead dominating total runtime above 90% sparsity.

7. References and Reproducibility Materials

- **[1]** T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022, ch. 4, pp. 75–89.
- **[2]** R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. Boston, MA, USA: Pearson, 2016, ch. 6, pp. 635–665.
- **[3]** V. Strassen, "Gaussian elimination is not optimal," *Numerische Mathematik*, vol. 13, no. 4, pp. 354–356, Aug. 1969.
- **[4]** Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA, USA: SIAM, 2003, ch. 3 (Sparse Matrices), pp. 75–102.

Reproducibility Package & AI Disclosure: Reused baseline (`src/baseline_matmul.py`), Assignment 2 implementations (`src/matmul_a2.py`), raw and summary CSV datasets (`data/raw_measurements.csv`, `data/summary_part_a_dense.csv`, `data/summary_part_b_sparse.csv`), and the automated Jupyter notebook (`notebooks/experiments_and_analysis.ipynb`) are available in the GitHub repository `Assignment02_Barreto`. *AI Disclosure:* Generative AI was used to assist in planning the experimental harness and structuring the English prose of this report; all code execution, measurements and conclusions were verified by the author.